Deep Agents 마스터 — 강사용 상세 대본

- 슬라이드 수: 231

- 부제: 장기 작업 에이전트와 운영 하네스

- 기준: source-repo/04_deepagents/*.ipynb

---

01. AI 에이전트 개발 실전

안녕하세요. AI 에이전트 개발 실전 과정을 시작하겠습니다. 이번 트랙의 중심은 계획, 파일시스템, 메모리, 서브에이전트, 스킬을 묶어 장기 작업 하네스를 설계하는 일입니다. 슬라이드는 개념의 흐름을 잡기 위한 자료이고, 뒤쪽 코드 부록은 실제 노트북의 실행 셀을 순서대로 확인하기 위한 자료입니다. 강의 중에는 API 이름을 외우는 것보다, 그 API가 어떤 문제를 해결하려고 등장했는지에 집중해 주세요. 이 기준을 잡으면 이후 실습에서 코드가 길어져도 실행 흐름을 놓치지 않습니다.

---

02. 오리엔테이션

이제 PART 01 오리엔테이션으로 넘어갑니다. LangChain 기초와 LangGraph 심화 위에 파일·서브에이전트·메모리·스킬을 얹고, 긴 작업에는 TodoListMiddleware로 계획을 추가합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다.

---

03. Deep Agents는 '에이전트 앱'이 아니라 장기 작업 하네스입니다

Deep Agents는 '에이전트 앱'이 아니라 장기 작업 하네스입니다. 최신 0.7 계열에서는 기본 하네스와 todo 계획 미들웨어를 구분해야 합니다. 파일, 백엔드, 서브에이전트, 스킬은 하네스의 중심 표면이고, todo 계획은 TodoListMiddleware를 붙여 활성화합니다.

---

04. Deep Agents 아키텍처는 LangGraph 위에 filesystem·subagents·memory와 선택 planning를 얹습니다

Deep Agents 아키텍처는 LangGraph 위에 filesystem, subagents, memory와 선택 planning을 얹습니다. 그림의 각 블록은 이후 노트북에서 코드로 직접 조정하는 표면입니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 Runtime는 LangGraph CompiledStateGraph로 실행됩니다. Middleware는 파일, 메모리, 스킬, 요약, 서브에이전트를 조합합니다. Backends는 상태·파일·스토어·샌드박스를 같은 파일 도구 표면으로 연결합니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

---

05. 04_deepagents는 11개 노트북으로 실전 하네스의 부품을 조립합니다

04_deepagents는 11개 노트북으로 실전 하네스의 부품을 조립합니다. 빠른 시작에서 끝나지 않고 백엔드, 서브에이전트, 샌드박스, ACP, 비동기 위임까지 갑니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 기본 생성은 01~03 소개·Quickstart·커스터마이징입니다. 판단 기준은 create_deep_agent와 시스템 프롬프트, 도구, 구조화 출력입니다. 작업 공간은 04~06 백엔드·서브에이전트·메모리/스킬입니다. 판단 기준은 파일·스토어·위임·AGENTS.md·SKILL.md 설계입니다. 고급 실행은 07~08 HITL·스트리밍·샌드박스·하네스입니다. 판단 기준은 승인, 코드 실행, 컨텍스트 관리, 프로필입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

06. Deep Agents 개요

이제 PART 02 Deep Agents 개요로 넘어갑니다. 아키텍처와 제품군, 핵심 개념 5가지를 먼저 고정합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 04_deepagents/01_introduction.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

07. Deep Agents 제품군은 같은 하네스를 서로 다른 실행 표면으로 제공합니다

Deep Agents 제품군은 같은 하네스를 서로 다른 실행 표면으로 제공합니다. Python API, CLI, ACP를 같은 개념의 다른 사용 표면으로 구분합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 Python API는 애플리케이션 코드에서 create_deep_agent 호출입니다. 판단 기준은 제품 안에 장기 작업 에이전트를 내장입니다. Deep Agents CLI는 터미널에서 파일 작업과 장기 작업 수행입니다. 판단 기준은 로컬 개발·문서 작업 자동화입니다. ACP는 에디터나 클라이언트가 에이전트를 표준 방식으로 호출입니다. 판단 기준은 Zed 같은 편집기 연동입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

08. 핵심 개념 5가지는 장기 작업 실패를 막기 위한 장치입니다

핵심 개념 5가지는 장기 작업 실패를 막기 위한 장치입니다. Planning, context, backend, subagent, memory/skill은 기능 목록이 아니라 실패 원인별 대응책입니다. 묶음별로 읽으면 긴 작업을 끝까지 밀고 가기에서는 Planning은 작업을 세부 단계로 나눕니다; Filesystem은 중간 산출물을 파일로 남깁니다; Context management는 불필요한 대화 누적을 줄입니다를 봅니다. 반복 작업을 안정화하기에서는 Backends는 상태·파일·메모리 저장 경계를 정합니다; Subagents는 전문 작업을 격리합니다; Skills는 절차 지식을 필요할 때 읽게 합니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

09. Deep Agents 제품군은 Python API, CLI, ACP 표면으로 나뉩니다

Deep Agents 제품군은 Python API, CLI, ACP 표면으로 나뉩니다. 같은 하네스 개념을 어디에서 실행하느냐에 따라 선택지가 달라집니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 Python API는 코드에서 create_deep_agent() 호출입니다. 판단 기준은 앱·서비스 안에 에이전트 내장입니다. Deep Agents CLI는 터미널에서 deepagents-cli 실행입니다. 판단 기준은 로컬 코딩·파일 작업입니다. ACP는 에디터의 Agent Client Protocol 호출입니다. 판단 기준은 Zed 같은 클라이언트에서 에이전트 사용입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

10. 핵심 개념 5가지는 장기 작업의 실패 원인을 막기 위한 장치입니다

핵심 개념은 장기 작업의 실패 원인을 막기 위한 장치입니다. 다만 planning은 항상 켜진 기능이라고 외우면 안 됩니다. 긴 작업에서 todo 상태가 필요할 때 TodoListMiddleware를 명시적으로 넣는 방식으로 설명합니다.

11. 설치 확인은 create_deep_agent와 핵심 타입 import로 시작합니다

설치 확인은 create_deep_agent와 핵심 타입 import로 시작합니다. Deep Agents는 LangGraph CompiledStateGraph를 반환하므로 LangGraph 실행 모델을 그대로 물려받습니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 화면에서 create_deep_agent는 실전 트랙의 기준 함수입니다. SubAgent는 동기 서브에이전트 정의에 사용합니다. CompiledSubAgent는 이미 컴파일된 그래프를 서브에이전트로 감쌀 때 사용합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 deepagents에서 create_deep_agent,

SubAgent, CompiledSubAgent; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 create_deep_agent, ChatOpenAI입니다. 노트북 기준 위치는 01_introduction.ipynb 노트북 기준입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

12. Deep Agents 기본 모델은 제품 기본값과 교재 기본값을 구분해야 합니다

Deep Agents 기본 모델은 제품 기본값과 교재 기본값을 구분해야 합니다. 노트북은 본 과정에서 OpenAI gpt-5.4를 쓰지만, Deep Agents 표준 기본은 anthropic:claude-sonnet-4-6로 설명합니다. 묶음별로 읽으면 교재 기본에서는 ChatOpenAI(model='gpt-5.4') 객체를 create_deep_agent에 넘깁니다; OPENAI_API_KEY를 기준으로 실습합니다를 봅니다. 프레임워크 기본에서는 모델 문자열 provider:model-name도 사용합니다; 서브에이전트마다 다른 모델을 지정합니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

---

13. Quickstart

이제 PART 03 Quickstart로 넘어갑니다. invoke, Tavily 검색 도구, 빌트인 도구, stream을 빠르게 체험합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 04_deepagents/02_quickstart.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

---

14. 초기 실행은 환경 변수와 모델 객체를 분리해 확인합니다

초기 실행은 환경 변수와 모델 객체를 분리해 확인합니다. 에이전트 생성 전에 API 키 로드와 모델 초기화를 따로 검증해야 오류 위치가 명확해집니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 화면에서 load_dotenv는 로컬 .env에서 키를 읽습니다. ChatOpenAI는 노트북 기준 모델 객체를 만듭니다. create_deep_agent는 모델 객체를 받아 LangGraph 실행 그래프를 반환합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 create_deep_agent, ChatOpenAI입니다. 노트북 기준 위치는 02_quickstart.ipynb 환경 확인입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

15. Quickstart에서 확인할 실행 표면은 invoke, tool, stream입니다

Quickstart에서 확인할 실행 표면은 invoke, tool, stream입니다. 처음에는 기능을 많이 붙이기보다 같은 에이전트를 세 방식으로 관찰합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 invoke는 최종 답변이 어떻게 반환되는지입니다. 판단 기준은 동기 요청/응답 기본값입니다. Tavily 도구는 일반 함수를 검색 도구로 연결하는 방식입니다. 판단 기준은 외부 정보 조회 권한입니다. Built-in tools는 계획과 파일 도구가 기본 제공되는지입니다. 판단 기준은 장기 작업 전제입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

---

16. 가장 작은 Deep Agent는 모델만 넘겨도 생성됩니다

가장 작은 Deep Agent는 모델만 넘겨도 생성됩니다. 반환 객체는 LangGraph 실행 그래프이므로 invoke와 stream을 그대로 사용합니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 화면에서 model은 노트북은 ChatOpenAI(model='gpt-5.4')를 사용합니다. invoke는 입력은 messages dict입니다. 반환은 마지막 메시지 content에서 답을 확인합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저

import는 deepagents에서 create_deep_agent; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 create_deep_agent, ChatOpenAI, invoke입니다. 노트북 기준 위치는 02_quickstart.ipynb 노트북 기준입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

17. Tavily 검색 도구를 붙이면 리서치 에이전트가 됩니다

Tavily 검색 도구를 붙이면 리서치 에이전트가 됩니다. 일반 함수도 타입 힌트와 docstring이 있으면 도구 설명으로 쓰입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 화면에서 검색 도구는 TavilyClient.search를 감싼 Python 함수입니다. system_prompt는 검색 후 한국어 보고서로 정리하라고 지시합니다. tools는 create_deep_agent의 tools 리스트로 전달합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 tavily에서 TavilyClient를 준비합니다. 함수 internet_search는 실제 작업 단위입니다. 핵심 API는 create_deep_agent입니다. 노트북 기준 위치는 02_quickstart.ipynb Tavily입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

18. 빌트인 표면은 파일·서브에이전트·스트리밍이고, 계획은 선택 미들웨어입니다

빌트인 표면을 설명할 때 계획을 항상 켜진 기능으로 고정하지 않습니다. Deep Agents 0.7 기준으로 todo 계획은 선택 미들웨어입니다. 파일, 서브에이전트, 스트리밍, 백엔드와 구분해서 봐야 버전이 올라가도 코드가 헷갈리지 않습니다.

---

19. stream()은 장기 작업 진행을 사용자에게 보여주는 기본 통로입니다

stream()은 장기 작업 진행을 사용자에게 보여주는 기본 통로입니다. 최종 답만 기다리지 않고 중간 메시지와 도구 사용을 관찰합니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 화면에서 updates는 노드별 변경분을 다룹니다. messages는 모델 토큰을 사용자에게 표시합니다. custom은 도구나 미들웨어가 보낸 진행률을 다룹니다. 코드는 다음 지점을 중심으로 읽습니다. 핵심 API는 stream입니다. 노트북 기준 위치는 02_quickstart.ipynb stream입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

20. 커스터마이징

이제 PART 04 커스터마이징으로 넘어갑니다. 모델 선택, 시스템 프롬프트, 커스텀 도구, response_format, 미들웨어 실행 순서를 정리합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 04_deepagents/03_customization.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

---

21. 커스텀 도구는 타입 힌트와 docstring으로 모델에게 사용법을 설명합니다

커스텀 도구는 타입 힌트와 docstring으로 모델에게 사용법을 설명합니다. 도구 설명이 부정확하면 모델은 언제 호출해야 하는지 판단하기 어렵습니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 화면에서 type hint는 입력 스키마의 기본이 됩니다. docstring는 모델이 도구 목적을 이해하는 설명입니다. return은 모델이 후속 추론에 사용할 관측값입니다. 코드는 다음 지점을 중심으로 읽습니다. 함수 calculate_total는 실제 작업 단위입니다. 핵심 API는 create_deep_agent입니다. 노트북 기준 위치는 03_customization.ipynb custom tool입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

22. 미들웨어 스택은 프롬프트 조립과 도구 실행 사이에 순서대로 개입합니다

미들웨어 스택은 프롬프트 조립과 도구 실행 사이에 순서대로 개입합니다. 커스터마이징은 옵션을 많이 켜는 일이 아니라 실행 경계를 좁히는 작업입니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 System prompt는 역할과 출력 정책을 고정합니다. 판단 기준은 system_prompt입니다. Tools는 허용된 행동 공간을 정합니다. 판단 기준은 tools=[...]입니다. Structured output는 최종 답변 형태를 강제합니다. 판단 기준은 response_format입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

23. 모델은 문자열 또는 ChatModel 객체로 전달할 수 있습니다

모델은 문자열 또는 ChatModel 객체로 전달합니다. 메인 에이전트와 서브에이전트의 모델을 다르게 잡는 것이 실전 비용 최적화의 시작입니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 ChatModel 객체는 ChatOpenAI(model='gpt-5.4')입니다. 판단 기준은 temperature, timeout 같은 provider 설정 직접 제어입니다. 모델 문자열은 openai:gpt-5.4입니다. 판단 기준은 설정 파일이나 CLI에서 바꾸기 쉬움입니다. 서브에이전트별 모델은 researcher는 mini, planner는 sonnet입니다. 판단 기준은 작업 난도에 맞춘 비용·지연 최적화입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

24. 시스템 프롬프트는 역할뿐 아니라 출력 형식과 위험 기준까지 고정합니다

시스템 프롬프트는 역할뿐 아니라 출력 형식과 위험 기준까지 고정합니다. 장기 작업 에이전트일수록 좋은 프롬프트는 정책 문서에 가깝습니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 화면에서 역할은 전문가 페르소나와 책임 범위를 고정합니다. 평가 기준은 보안·성능·가독성처럼 검토 축을 명시합니다. 응답 형식은 심각도나 섹션 구조를 고정해 후처리를 쉽게 만듭니다. 코드는 다음 지점을 중심으로 읽습니다. 핵심 API는 create_deep_agent입니다. 노트북 기준 위치는 03_customization.ipynb system_prompt입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

25. response_format은 최종 답을 Pydantic 스키마로 강제합니다

response_format은 최종 답을 Pydantic 스키마로 강제합니다. 보고서, 추천, 추출 결과처럼 후속 코드가 읽어야 하는 답변에 사용합니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 화면에서 BaseModel은 최종 응답 구조를 필드 단위로 정의합니다. Field는 모델이 각 필드의 의미를 이해하도록 설명합니다. response_format는 create_deep_agent에 스키마를 전달합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 pydantic에서 BaseModel, Field를 준비합니다. 클래스 BookRecommendation, BookRecommendationList는 데이터 구조나 정책 단위를 정의합니다. 핵심 API는 create_deep_agent입니다. 노트북 기준 위치는 03_customization.ipynb response_format입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

26. 미들웨어는 시스템 프롬프트 조립과 도구 실행 사이에 순서대로 개입합니다

미들웨어는 시스템 프롬프트 조립과 도구 실행 사이에 순서대로 개입합니다. Filesystem, Memory, SubAgent, Skills, Summarization 미들웨어가 기본 스택을 이룹니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 Prompt assembly는 메모리와 스킬이 시스템 컨텍스트에 추가됩니다. Tool surface는 파일·서브에이전트 도구가 실행 표면에 노출됩니다. Summarization은 긴 대화에서 컨텍스트를 압축합니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

27. 백엔드

이제 PART 05 백엔드로 넘어갑니다. StateBackend, FilesystemBackend, StoreBackend, CompositeBackend, LocalShellBackend, custom backend를 정리합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 04_deepagents/04_backends.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

28. 백엔드 선택은 저장 위치보다 보안 경계와 수명 주기를 먼저 봐야 합니다

백엔드는 저장 위치보다 보안 경계와 수명 주기를 먼저 봅니다. 최신 계약에서는 StateBackend는 인자 없이 만들고, StoreBackend는 namespace 정책을 분명히 둡니다. 위치 runtime 인자를 넘기는 오래된 예시는 더 이상 기준으로 삼지 않습니다.

29. FilesystemBackend를 쓸 때는 읽기·쓰기 범위를 운영 정책으로 제한해야 합니다

FilesystemBackend를 쓸 때는 읽기·쓰기 범위를 운영 정책으로 제한해야 합니다. 로컬 파일 접근은 강력하지만 잘못 열면 에이전트가 불필요한 파일까지 읽을 수 있습니다. 묶음별로 읽으면 필수 점검에서는 root_dir를 프로젝트 작업 폴더로 제한합니다; 가상 모드와 실제 디스크 쓰기의 차이를 설명합니다; API 키와 개인정보 파일을 작업 경로에서 분리합니다를 봅니다. 운영 기준에서는 임시 산출물과 장기 메모리 경로를 나눕니다; 삭제 요청이 들어왔을 때 어떤 저장소를 지워야 하는지 정합니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

30. Backend abstraction은 파일 도구를 실제 저장소와 분리합니다

Backend abstraction은 파일 도구를 실제 저장소와 분리합니다. 에이전트는 같은 read/write/edit 표면을 쓰지만 뒤쪽 저장소는 상태, 디스크, 스토어, 샌드박스가 됩니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 StateBackend는 기본값이며 에이전트 state 안에 파일을 저장합니다. FilesystemBackend는 root_dir 안의 로컬 디스크를 사용합니다. StoreBackend는 LangGraph Store로 thread 간 장기 메모리를 공유합니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

31. 백엔드 선택은 보안 경계와 영속성 요구를 같이 봐야 합니다

백엔드 선택은 보안 경계와 영속성 요구를 같이 봐야 합니다. 파일을 어디에 쓰는지보다 누가 읽을 수 있고 언제 사라지는지가 더 중요합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 StateBackend는 설정 없이 바로 사용입니다. 판단 기준은 세션 state 중심, 장기 저장에 한계입니다. FilesystemBackend는 실제 파일과 호환입니다. 판단 기준은 root_dir, virtual_mode로 경계 제한 필요하다는 점입니다. StoreBackend는 사용자별 장기 메모리입니다. 판단 기준은 namespace 정책이 핵심입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

32. Backend 0.7 계약은 생성자와 반환 형식을 따로 검수합니다

Backend 0.7 계약은 생성자와 반환 형식을 따로 검수합니다. StateBackend는 인자 없이 시작하고, StoreBackend는 namespace 정책을 명시해야 합니다. 또 도구가 사용자에게 보여주는 문자열과 backend가 반환하는 구조화된 값은 다릅니다. 기존 helper 이름이나 출력 문자열에 의존한 설명은 운영 코드로 옮기면 위험합니다.

33. 파일 도구는 virtual_mode, overwrite, delete 권한을 함께 봅니다

파일 도구는 저장 위치만 보지 않습니다. virtual_mode가 실제 파일 경계를 어떻게 가리는지, write_file이 기존 파일을 교체할 수 있는지, backend가 delete를 노출하는지까지 봐야 합니다. 그래서 permissions 수업에서는 쓰기 권한과 삭제 권한을 같은 위험군으로 다룹니다.

34. fork 모드와 skill tools는 같은 위임 기능으로 묶어 외우면 안 됩니다

서브에이전트 위임과 스킬 도구 노출은 서로 다른 계약입니다. 기본 isolated 위임은 필요한 입력만 넘겨 메인 컨텍스트를 보호합니다. fork 모드는 부모 대화 문맥을 이어받는 실험적 모드이고, skill의 include_tools나 pinned_skills는 도구와 절차 노출을 제어하는 별도 표면입니다.

35. 기본 미들웨어 대체는 같은 이름의 인스턴스로 의도를 드러냅니다

Deep Agents의 기본 미들웨어를 바꿀 때는 같은 이름을 가진 인스턴스로 교체 의도를 드러냅니다. 새 정책을 추가하는 것과 기존 요약, 파일, 스킬 미들웨어를 대체하는 것은 다릅니다. 이 차이를 알아야 디버깅할 때 어느 정책이 실제로 실행됐는지 찾을 수 있습니다.

36. Rubric과 grader hook은 교육용 채점표와 SDK 실행 계약을 구분합니다

Rubric과 grader hook은 수업용 채점표와 SDK 실행 계약을 구분해야 합니다. 기준 문장만 쓰면 좋아 보이지만, 실제 운영에서는 trace, 최종 답변, 도구 trajectory 중 무엇을 evidence로 받을지 정해야 합니다. 점수뿐 아니라 실패 원인을 남겨야 다음 실험으로 이어집니다.

37. CompositeBackend는 경로 prefix로 저장소를 라우팅합니다

CompositeBackend는 경로 prefix로 저장소를 라우팅합니다. /memories/는 StoreBackend로, 나머지는 StateBackend로 보내는 식의 분리가 가능합니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 default는 명시 라우트가 없는 파일을 처리합니다. routes는 경로 prefix별 백엔드를 지정합니다. policy는 영속 파일과 임시 파일을 코드가 아니라 경로 정책으로 나눕니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

38. CompositeBackend는 장기 메모리와 임시 작업 파일을 경로로 나눕니다

CompositeBackend는 장기 메모리와 임시 작업 파일을 경로로 나눕니다. Store와 checkpointer를 같이 넘겨야 메모리와 실행 상태가 모두 유지됩니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 화면에서 StoreBackend는 /memories/ 아래 파일만 장기 store로 보냅니다. StateBackend는 나머지 작업 파일은 state에 둡니다. namespace는 운영에서는 runtime context의 user_id를 사용합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 backends에서 StateBackend, StoreBackend, CompositeBackend; memory에서 InMemoryStore를 준비합니다. 핵심 API는 create_deep_agent, InMemoryStore입니다. 노트북 기준 위치는 04_backends.ipynb CompositeBackend입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

39. 서브에이전트

이제 PART 06 서브에이전트로 넘어갑니다. dict 기반 SubAgent, CompiledSubAgent, general-purpose 오버라이드, 컨텍스트 전파, 멀티 파이프라인을 다룹니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 04_deepagents/05_subagents.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

40. SubAgent 정의의 핵심은 name보다 description입니다

SubAgent 정의의 핵심은 name보다 description입니다. 라우터는 description을 읽고 어떤 작업을 맡길지 판단합니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 화면에서 name는 도구로 노출되는 역할 이름입니다. description은 호출 조건을 설명하는 라우팅 문장입니다. system_prompt는 서브에이전트 내부 역할과 출력 기준입니다. 코드는 다음 지점을 중심으로 읽습니다. 핵심 API는 create_deep_agent입니다. 노트북 기준 위치는 05_subagents.ipynb SubAgent dict입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

41. 서브에이전트 상태 관리는 상속과 독립 체크포인터를 구분해야 합니다

서브에이전트 상태 관리는 상속과 독립 체크포인터를 구분해야 합니다. 전문 역할을 나누는 것과 대화 히스토리를 어떻게 보존하는지는 별도 결정입니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 기본 상속은 부모 실행 컨텍스트 안에서 동작합니다. 판단 기준은 간단한 전문 작업 위임입니다. checkpointer=True는 서브에이전트 자체 히스토리를 유지합니다. 판단 기준은 장기 전문 대화가 필요한 역할입니다. CompiledSubAgent는 이미 컴파일된 그래프를 서브에이전트로 사용합니다. 판단 기준은 전문 워크플로 재사용입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

42. 서브에이전트는 메인 대화 컨텍스트를 보호하면서 전문 작업을 맡깁니다

서브에이전트는 메인 대화 컨텍스트를 보호하면서 전문 작업을 맡깁니다. 긴 리서치나 분석 결과는 요약된 결과만 메인 에이전트로 돌아옵니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 메인 에이전트는 사용자 요청을 분석하고 위임 여부를 결정합니다. 전문 서브에이전트는 자기 도구와 시스템 프롬프트로 깊은 작업을 수행합니다. 결과 합성은 메인 에이전트가 결과를 최종 답변으로 통합합니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

43. dict 기반 SubAgent는 name, description, system_prompt, tools로 정의합니다

dict 기반 SubAgent는 name, description, system_prompt, tools로 정의합니다. description은 라우터가 언제 이 역할을 호출할지 판단하는 핵심 문장입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 화면에서 name는 도구처럼 호출될 고유 이름입니다. description은 위임 기준을 분명하게 적습니다. tools는 서브에이전트에게만 필요한 최소 도구를 줍니다. 코드는 다음 지점을 중심으로 읽습니다. 핵심 API는 create_deep_agent입니다. 노트북 기준 위치는 05_subagents.ipynb SubAgent입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

44. 멀티 서브에이전트 파이프라인은 수집·분석·작성 책임을 분리합니다

멀티 서브에이전트 파이프라인은 수집·분석·작성 책임을 분리합니다. 각 역할이 자기 산출을 짧게 반환해야 메인 컨텍스트가 부풀지 않습니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 data-collector은 원시 자료를 모읍니다. data-analyzer은 패턴과 인사이트를 추출합니다. report-writer은 최종 보고서 구조로 정리합니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

45. 좋은 서브에이전트 설계는 역할을 많이 만드는 것이 아니라 경계를 선명하게 하는 것입니다

서브에이전트 설계에서는 기본 격리 모드와 fork 모드를 구분합니다. fork는 부모 대화 문맥을 이어받는 실험적 모드이고, 별도 skills 설정과 함께 쓰지 않습니다. 모든 서브에이전트가 항상 부모와 완전히 단절된다고 설명하면 이제 부족합니다.

46. 메모리와 스킬

이제 PART 07 메모리와 스킬으로 넘어갑니다. 장기 메모리, AGENTS.md, Skills, Progressive Disclosure, 스킬 소스 우선순위를 정리합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 04_deepagents/06_memory_and_skills.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

47. CompositeBackend는 임시 파일과 장기 메모리를 경로 prefix로 나눕니다

CompositeBackend는 임시 파일과 장기 메모리를 경로 prefix로 나눕니다. 일반 작업 파일은 StateBackend로 보내고, /memories/ 아래 파일은 StoreBackend로 보냅니다. 데모에서는 고정 namespace를 명시하고, 운영에서는 검증된 context_schema로 사용자 경계를 연결합니다.

48. 메모리는 사실을 저장하고 스킬은 절차를 저장합니다

메모리는 사실을 저장하고 스킬은 절차를 저장합니다. 둘을 섞으면 에이전트가 무엇을 기억하고 무엇을 따라야 하는지 불분명해집니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 Memory는 사용자 선호, 프로젝트 사실, 장기 상태입니다. 판단 기준은 /memories/입니다. AGENTS.md는 프로젝트 전체 작업 계약과 제약입니다. 판단 기준은 memory 파라미터입니다. SKILL.md는 특정 작업을 수행하는 절차 지침입니다. 판단 기준은 skills 디렉터리입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

49. 메모리는 사실을 저장하고 스킬은 절차를 저장합니다

메모리는 사실을 저장하고 스킬은 절차를 저장합니다. 둘을 섞으면 에이전트가 무엇을 기억해야 하고 무엇을 따라야 하는지 흐려집니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 Memory는 사용자 선호, 프로젝트 상태, 장기 사실입니다. 판단 기준은 선호 언어, 고객명, 이전 결정입니다. Skills는 반복 가능한 절차와 도구 사용법입니다. 판단 기준은 리서치 절차, 코드 리뷰 방식, 배포 체크입니다. AGENTS.md는 프로젝트 운영 규약입니다. 판단 기준은 폴더 규칙, 테스트 명령, 금지 사항입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

50. CompositeBackend는 /memories/ 경로만 장기 저장소로 보냅니다

CompositeBackend는 /memories/ 경로만 장기 저장소로 보냅니다. 사용자가 기억해 달라는 정보와 임시 파일을 저장 경로로 분리합니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 화면에서 store는 thread를 넘는 장기 저장소입니다. checkpointer은 현재 에이전트 실행 상태를 유지합니다. backend는 파일 경로를 StoreBackend와 StateBackend로 라우팅합니다. 코드는 다음 지점을 중심으로 읽습니다. 핵심 API는 create_deep_agent, InMemoryStore입니다. 노트북 기준 위치는 06_memory_and_skills.ipynb Memory backend입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

51. Progressive Disclosure는 필요한 순간에만 스킬 세부 지침을 읽게 합니다

Progressive Disclosure는 필요한 순간에만 스킬 세부 지침을 읽게 합니다. 처음부터 모든 절차를 시스템 프롬프트에 넣으면 컨텍스트가 낭비되고 충돌 가능성이 커집니다. 묶음별로 읽으면 SKILL.md 구조에서는 언제 사용할지, 어떤 워크플로를 따를지, 어떤 파일을 더 읽을지 적습니다; scripts와 assets가 있으면 직접 재작성보다 재사용하도록 안내합니다를 봅니다. 스킬 소스 우선순위에서는 프로젝트 로컬 스킬이 가장 구체적입니다; 사용자 홈 스킬과 번들 스킬은 더 넓은 기본 지침입니다를 봅니다. 서브에이전트 상속에서는 메인 에이전트가 가진 스킬을 그대로 물려받을지, 특정 역할에만 줄지 결정합니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

52. AGENTS.md는 memory 파라미터로 에이전트 컨텍스트에 주입할 수 있습니다

AGENTS.md와 스킬은 컨텍스트와 실행 입력을 구분해서 봅니다. memory는 읽어 넣을 파일 경로이고, 특정 스킬을 미리 열어야 할 때는 실행 입력 state의 pinned_skills로 전달합니다. include_tools는 SKILL.md metadata 안에서 공백으로 나열하는 도구 이름 문자열입니다.

53. 고급 기능

이제 PART 08 고급 기능으로 넘어갑니다. HITL decision 타입, v2 streaming, sandbox, CodeInterpreterMiddleware, ACP, CLI를 한데 묶습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 04_deepagents/07_advanced.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

54. HITL은 interrupt_on과 checkpointer가 함께 있어야 재개할 수 있습니다

HITL은 interrupt_on과 checkpointer가 함께 있어야 재개합니다. 위험 도구를 멈추는 것만으로는 부족하고, 중단 지점을 저장해야 사람이 결정한 뒤 이어갈 수 있습니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 화면에서 interrupt_on은 승인이 필요한 도구를 지정합니다. checkpointer은 중단 지점을 저장합니다. Command(resume)는 사람의 결정을 넣어 재개합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 memory에서 InMemorySaver를 준비합니다. 핵심 API는 create_deep_agent, InMemorySaver입니다. 노트북 기준 위치는 07_advanced.ipynb HITL입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

55. 고급 기능은 모두 권한 경계와 관찰 가능성을 함께 요구합니다

고급 기능은 모두 권한 경계와 관찰 가능성을 함께 요구합니다. 스트리밍, 샌드박스, 인터프리터, ACP는 기능 추가가 아니라 운영 경계 확장입니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 v2 Streaming는 도구 호출과 메시지 진행 관찰입니다. 판단 기준은 UI 표시와 로그 포맷입니다. Sandbox는 코드 실행 격리입니다. 판단 기준은 시크릿 반입 금지와 파일 범위입니다. Code Interpreter은 분석 코드 실행입니다. 판단 기준은 실행 시간과 패키지 제한입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

56. HITL은 위험한 도구 호출을 사람 승인 지점으로 바꿉니다

HITL은 위험한 도구 호출을 사람 승인 지점으로 바꿉니다. write_file과 edit_file 같은 도구는 interrupt_on으로 사전 승인을 걸 수 있습니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 interrupt_on은 승인이 필요한 도구 이름과 정책을 지정합니다. checkpointer은 승인 전 상태를 저장하기 위해 필수입니다. decisions는 approve, edit, reject, respond 네 타입으로 재개합니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

57. HITL 재개는 Command(resume={'decisions': [...]}) 형식으로 처리합니다

HITL 재개는 Command(resume={'decisions': [...]}) 형식으로 처리합니다. 인터럽트 개수와 같은 순서로 decisions 배열을 넘겨야 합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 화면에서 approve는 도구 호출을 그대로 실행합니다. edit는 도구 이름과 인자를 사람이 수정합니다. reject/respond는 실행을 막거나 사람 메시지로 대체합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 types에서 Command를 준비합니다. 핵심 API는 invoke, Command입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. 노트북 기준 위치는 07_advanced.ipynb Command decisions입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

58. Sandbox-as-Tool은 에이전트를 밖에 두고 실행 환경만 격리합니다

Sandbox-as-Tool은 에이전트를 밖에 두고 실행 환경만 격리합니다. 노트북은 Agent-in-Sandbox보다 Sandbox-as-Tool 패턴을 권장 관점으로 설명합니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 Agent는 모델과 orchestration은 통제 가능한 환경에 둡니다. Sandbox는 파일 실행과 코드 테스트를 격리합니다. Lifecycle는 작업 후 terminate 등 리소스 정리가 필수입니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

59. 고급 기능은 모두 권한 경계와 함께 봐야 합니다

고급 기능은 모두 권한 경계와 함께 봐야 합니다. 도구를 더 많이 주는 것보다 어떤 요청에서 멈출지 정하는 것이 운영의 핵심입니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 v2 Streaming는 subagent와 tool 진행 관찰입니다. 판단 기준은 namespace와 custom 이벤트 설계입니다. Sandbox는 코드 실행 격리입니다. 판단 기준은 시크릿을 샌드박스에 넣지 않음입니다. CodeInterpreterMiddleware는 QuickJS 기반 코드 실행입니다. 판단 기준은 PTC allowlist와 timeout 설정입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

60. AgentHarness

이제 PART 09 AgentHarness로 넘어갑니다. 계획 도구, 가상 파일시스템, 컨텍스트 관리, 코드 실행, HITL, 스킬과 메모리, 프로필을 정리합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 04_deepagents/08_harness.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

61. AgentHarness는 장기 작업 산출물을 만들기 위한 실행 셸입니다

AgentHarness는 계획, 파일, 위임, 실행을 하나의 작업 환경으로 묶습니다. 여기서 계획 도구는 TodoListMiddleware를 붙인 경우에 활성화됩니다. 이 차이를 분명히 해야 최신 Deep Agents 코드와 슬라이드 설명이 맞습니다.

62. 하네스 설계는 컨텍스트를 많이 넣는 일이 아니라 회수 가능하게 나누는 일입니다

하네스 설계는 컨텍스트를 많이 넣는 일이 아니라 회수 가능하게 나누는 일입니다. 장기 작업에서는 정보가 시스템 프롬프트, 파일, 메모리, 스킬 중 어디에 둘지 계속 분리해야 합니다. 묶음별로 읽으면 시스템 프롬프트에 둘 것에서는 역할, 금지 행동, 출력 계약; 실행 전체에 항상 필요한 정책를 봅니다. 파일/메모리에 둘 것에서는 중간 산출물과 재사용할 근거 자료; 사용자 또는 프로젝트별 장기 사실를 봅니다. 스킬에 둘 것에서는 반복 가능한 절차와 도구 사용 순서; 필요할 때만 읽어도 되는 세부 지침를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

63. 하네스는 에이전트에게 작업 공간과 운영 규칙을 함께 제공합니다

하네스는 에이전트에게 작업 공간과 운영 규칙을 함께 제공합니다. 단일 agent.invoke가 아니라 장기 작업을 끝까지 밀고 가는 실행 환경으로 봐야 합니다. 핵심 레이어는 다음과 같습니다. Planning Tool은 계획에 해당합니다. 할 일 목록을 만들고 진행 상태를 갱신합니다. 코드에서는 write_todos 같은 형태로 드러납니다. Virtual FS는 파일에 해당합니다. read_file, write_file, edit_file로 작업 산출물을 다룹니다. 코드에서는 BackendProtocol 같은 형태로 드러납니다. Delegation은 위임에 해당합니다. 서브에이전트와 비동기 작업으로 긴 작업을 나눕니다. 코드에서는 SubAgent 같은 형태로 드러납니다. Execution은 코드에 해당합니다. 샌드박스나 interpreter로 코드를 실행합니다. 코드에서는 execute · QuickJS 같은 형태로 드러납니다. 이 구조를 잡아 두면 뒤에서 API가 늘어나도 같은 기준으로 분류합니다.

64. 하네스 기능은 사용자 요청을 산출물 중심 작업으로 바꾸는 데 초점이 있습니다

하네스 기능은 사용자 요청을 산출물 중심 작업으로 바꾸는 데 초점이 있습니다. 계획, 파일, 코드 실행이 결합되어야 장기 작업의 중간 상태가 남습니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 계획 도구는 작업을 단계화입니다. 판단 기준은 완료·미완료가 드러남입니다. 가상 파일시스템은 산출물 저장입니다. 판단 기준은 파일 diff와 결과물 확인입니다. 컨텍스트 조립은 입력 규약·메모리·스킬 결합입니다. 판단 기준은 필요한 지침만 주입입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

65. 프로필은 모델과 도구, 권한을 실행 맥락별로 묶는 설정 단위입니다

프로필은 모델과 도구, 권한을 실행 맥락별로 묶는 설정 단위입니다. 같은 하네스라도 로컬 개발, 리서치, 코드 리뷰 프로필의 권한은 달라야 합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 화면에서 model은 프로필별 기본 모델을 고정합니다. tools는 허용 도구와 백엔드를 제한합니다. permissions는 파일 쓰기와 셸 실행 같은 위험 권한을 분리합니다. 노트북 기준 위치는 08_harness.ipynb Profiles입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

66. 하네스 설계의 핵심은 컨텍스트를 많이 주는 것이 아니라 회수 가능하게 주는 것입니다

하네스 설계의 핵심은 컨텍스트를 많이 주는 것이 아니라 회수 가능하게 주는 것입니다. 장기 작업에서는 어떤 정보가 시스템 프롬프트, 메모리, 파일, 스킬에 둘지 계속 분리해야 합니다. 묶음별로 읽으면 입력 컨텍스트 조립에서는 요청, AGENTS.md, memory, skill 지침을 역할별로 조립합니다; 서브에이전트에는 필요한 일부만 전파합니다를 봅니다. 런타임 컨텍스트 압축에서는 대화가 길어질 때 요약은 작업 의도와 완료 기준을 보존해야 합니다; 파일 산출물은 대화보다 더 안정적인 상태 저장소가 됩니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

67. 프레임워크 비교

이제 PART 10 프레임워크 비교로 넘어갑니다. Deep Agents, OpenCode, Claude Agent SDK의 차이와 사용 사례별 추천을 정리합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 04_deepagents/09_comparison.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

68. Deep Agents, OpenCode, Claude Agent SDK는 실행 위치와 내장 방식이 다릅니다

Deep Agents, OpenCode, Claude Agent SDK는 실행 위치와 내장 방식이 다릅니다. 비교는 기능 개수가 아니라 누가 어디서 어떤 권한으로 실행하는지를 기준으로 봅니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 실행 모델은 앱 안에 내장할 것인가, CLI에서 쓸 것인가입니다. 판단 기준은 제품 내장 vs 로컬 작업입니다. 멀티테넌시는 사용자별 권한과 상태가 필요한가입니다. 판단 기준은 서비스 운영 요구입니다. 모델 선택은 여러 provider와 모델을 바꿀 수 있는가입니다. 판단 기준은 비용과 품질 최적화입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

69. 프레임워크 선택은 사용 사례 문장으로 결정해야 합니다

프레임워크 선택은 사용 사례 문장으로 결정해야 합니다. 기술 이름보다 실행 맥락을 먼저 쓰면 선택지가 좁혀집니다. 카드별로 보면 로컬 개발 보조는 개발자 개인 작업과 파일 편집이 중심입니다. 제품 내장 에이전트는 사용자별 상태, 권한, 과금, 로그가 필요합니다. 에디터 연동은 표준 프로토콜로 에이전트를 클라이언트에 붙입니다. 장기 업무 자동화는 계획, 파일, 서브에이전트, 메모리가 함께 필요합니다. 이 슬라이드는 선택지를 나열하는 장면입니다. 실제 설계에서는 지금 보이는 항목 중 무엇을 기본값으로 둘지, 무엇을 나중에 확장할지 결정해야 합니다.

70. Deep Agents, OpenCode, Claude Agent SDK는 같은 '코딩 에이전트'가 아닙니다

Deep Agents, OpenCode, Claude Agent SDK는 같은 '코딩 에이전트'가 아닙니다. 비교 기준은 기능 개수보다 실행 모델, 배포, 멀티테넌시, 모델 설정입니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 실행 모델은 Python/LangGraph 앱 안에 내장입니다. 판단 기준은 CLI 또는 SDK 중심 실행입니다. 배포는 서비스·멀티테넌트 구성에 유리입니다. 판단 기준은 개발자 로컬·특정 플랫폼에 강점입니다. 모델 설정은 프로바이더 문자열과 ChatModel 객체입니다. 판단 기준은 도구별 설정 방식이 다름입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

71. 사용 사례별 추천은 '어디에서 누가 실행하나'로 정리됩니다

사용 사례별 추천은 '어디에서 누가 실행하나'로 정리됩니다. 로컬 개발자 보조와 제품 내장 에이전트는 같은 선택지가 아닙니다. 흐름을 왼쪽에서 오른쪽으로 읽으면 LOCAL 단계에서는 개발자 개인를 다룹니다. APP 단계에서는 제품 내장를 다룹니다. EDITOR 단계에서는 에디터 통합를 다룹니다. TEAM 단계에서는 운영 거버넌스를 다룹니다. 프레임워크 선택은 기능 데모가 아니라 운영 위치와 책임 경계의 결정입니다. 에이전트 수업에서 중요한 것은 각 단계를 별개의 기능으로 보는 것이 아니라, 입력과 출력이 다음 단계로 어떻게 넘어가는지 보는 것입니다.

72. 마이그레이션은 도구 목록보다 메모리와 파일 계약을 먼저 옮겨야 합니다

마이그레이션은 도구 목록보다 메모리와 파일 계약을 먼저 옮겨야 합니다. 장기 작업 에이전트의 핵심 상태는 프롬프트가 아니라 산출물, 메모리, 권한 정책에 있습니다. 묶음별로 읽으면 공통 고려사항에서는 도구 이름과 입력 스키마가 바뀌면 모델 호출 품질이 달라집니다; 파일 경로와 작업 디렉터리 규칙은 사용자 경험에 직접 영향을 줍니다를 봅니다. Deep Agents로 옮길 때 장점에서는 LangGraph 실행·체크포인터·스토어를 그대로 활용합니다; SubAgent와 backend를 앱 요구사항에 맞게 조합합니다를 봅니다. 주의사항에서는 무제한 파일/셸 권한을 기본값처럼 열면 보안 경계가 무너집니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

73. 선택 질문은 네 가지로 압축할 수 있습니다

선택 질문은 네 가지로 압축합니다. 답이 선명하면 프레임워크 논쟁보다 구현 경계가 먼저 정해집니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 제품 안에 넣을 것인가는 예, 사용자별 상태와 배포가 필요하다는 점입니다. 판단 기준은 아니오, 개인 로컬 자동화 중심입니다. 멀티테넌시가 필요한가는 사용자별 namespace와 store 필요하다는 점입니다. 판단 기준은 단일 개발자 세션입니다. 파일·메모리 계약이 복잡한가는 백엔드 라우팅이 필요하다는 점입니다. 판단 기준은 일회성 코드 작업입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

74. 샌드박스와 ACP

이제 PART 11 샌드박스와 ACP로 넘어갑니다. Sandbox-as-Tool, 보안 가이드라인, 파일 라이프사이클, ACP 서버와 에디터 연동을 정리합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 04_deepagents/10_sandboxes_and_acp.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

75. Sandbox-as-Tool은 에이전트 제어면과 실행 환경을 분리합니다

Sandbox-as-Tool은 에이전트 제어면과 실행 환경을 분리합니다. 에이전트를 통째로 샌드박스에 넣는 것보다 실행만 격리하는 패턴을 먼저 검토합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 Agent-in-Sandbox는 에이전트 전체를 격리 환경 안에서 실행입니다. 판단 기준은 시크릿과 제어면 관리가 복잡합니다. Sandbox-as-Tool은 에이전트는 밖에 두고 실행 도구만 격리입니다. 판단 기준은 권장 패턴으로 설명됩니다. 파일 전송은 필요한 입력 파일만 샌드박스로 전달입니다. 판단 기준은 민감 파일 반입 금지입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

76. ACP는 도구 서버가 아니라 에이전트를 클라이언트에 노출하는 연결 표면입니다

ACP는 도구 서버가 아니라 에이전트를 클라이언트에 노출하는 연결 표면입니다. MCP와 ACP를 혼동하면 도구 서버를 만들 곳에 에이전트 서버를 만들거나 반대로 설계합니다. 묶음별로 읽으면 MCP에서는 외부 도구와 데이터 소스를 모델/에이전트에 연결합니다; 서버는 도구를 제공하고 클라이언트가 호출합니다를 봅니다. ACP에서는 에디터나 앱이 에이전트를 클라이언트처럼 호출합니다; 세션, 메시지, 실행 결과를 에이전트 프로토콜로 다룹니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

77. Sandbox-as-Tool은 실행 환경을 격리하고 에이전트 제어면은 밖에 둡니다

Sandbox-as-Tool은 실행 환경을 격리하고 에이전트 제어면은 밖에 둡니다. 시크릿과 정책은 에이전트 쪽에 남기고, 샌드박스에는 실행에 필요한 최소 파일만 보냅니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 격리는 코드 실행과 파일 변경을 별도 환경에 둡니다. 전송은 필요한 파일만 업로드하고 결과만 회수합니다. 정리는 작업 후 샌드박스 리소스를 종료합니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

78. 샌드박스 보안은 '격리했으니 안전하다'가 아니라 '무엇을 넣지 않을지'가 핵심입니다

샌드박스 보안은 '격리했으니 안전하다'가 아니라 '무엇을 넣지 않을지'가 핵심입니다. 노트북은 시크릿을 샌드박스 안에 넣지 말라고 강조합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 시크릿 금지는 API 키는 샌드박스 밖에서 관리입니다. 판단 기준은 실행 환경 탈취 시 키 노출입니다. 파일 최소화는 필요한 입력만 업로드입니다. 판단 기준은 불필요한 내부 자료 유출입니다. 라이프사이클은 작업 후 terminate입니다. 판단 기준은 비용 증가와 잔류 프로세스입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

79. ACP 서버는 에디터가 에이전트를 표준 클라이언트처럼 호출하게 합니다

ACP 서버는 에디터가 에이전트를 표준 클라이언트처럼 호출하게 합니다. MCP가 도구 서버라면 ACP는 에이전트 클라이언트 프로토콜입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 화면에서 AgentServerACP는 Deep Agent를 ACP 서버로 감쌉니다. run_agent는 ACP 런타임에서 서버를 실행합니다. 에디터는 Zed나 Toad 같은 클라이언트가 연결합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 acp에서 run_agent; deepagents에서 create_deep_agent를 준비합니다. 함수 main는 실제 작업 단위입니다. 핵심 API는 create_deep_agent입니다. 노트북 기준 위치는 10_sandboxes_and_acp.ipynb ACP server입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

80. MCP와 ACP는 이름이 비슷하지만 연결 방향이 다릅니다

MCP와 ACP는 이름이 비슷하지만 연결 방향이 다릅니다. 혼동하면 도구 서버를 만들어야 할 곳에 에이전트 서버를 만들거나 반대로 설계합니다. 묶음별로 읽으면 MCP에서는 모델 또는 에이전트가 사용할 도구와 리소스를 외부 서버로 노출합니다; 검색, DB, 파일 시스템 같은 기능 제공에 적합합니다를 봅니다. ACP에서는 에디터나 클라이언트가 에이전트 자체와 대화하는 프로토콜입니다; 코딩 에이전트 UX와 장기 작업 세션에 적합합니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

81. 비동기 서브에이전트

이제 PART 12 비동기 서브에이전트로 넘어갑니다. AsyncSubAgent, supervisor, 5가지 관리 도구, langgraph.json 공동 배포, v2 streaming 관찰을 정리합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 04_deepagents/11_async_subagents.ipynb입니다. 슬라이드만 넘기지 말

고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

---

82. AsyncSubAgent는 오래 걸리는 작업을 별도 그래프로 시작하게 합니다

AsyncSubAgent는 오래 걸리는 작업을 별도 그래프로 시작하게 합니다. 사용자에게 즉시 제어를 돌려주고 배경 작업 상태를 나중에 확인해야 할 때 사용합니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 화면에서 name는 슈퍼바이저가 호출할 작업 이름입니다. graph_id/url은 같은 배포의 그래프 또는 원격 그래프를 지정합니다. tools는 슈퍼바이저에게 관리 도구가 노출됩니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 deepagents에서 AsyncSubAgent, create_deep_agent를 준비합니다. 핵심 API는 create_deep_agent입니다. 노트북 기준 위치는 11_async_subagents.ipynb AsyncSubAgent입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

83. Deep Agents는 filesystem, subagents, memory와 선택 planning을 묶은 agent harness입니다

슈퍼바이저는 비동기 작업을 관리하는 5가지 도구를 사용합니다. 비동기 위임은 시작보다 상태 확인과 취소 정책이 더 중요합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 create_async_task는 새 배경 작업을 시작합니다. 판단 기준은 즉시 결과를 기다리지 않습니다. check_async_task는 작업 상태와 결과를 확인합니다. 판단 기준은 사용자에게 진행 상황을 설명합니다. update_async_task는 작업 지시를 보강합니다. 판단 기준은 장기 작업의 요구 변경 대응입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

---

84. AsyncSubAgent는 즉시 끝나지 않는 작업을 별도 그래프로 맡깁니다

AsyncSubAgent는 즉시 끝나지 않는 작업을 별도 그래프로 맡깁니다. 공동 ASGI 배포에서는 url 없이 graph_id로 연결하고, 원격이면 url을 지정합니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 화면에서 graph_id는 배경 작업을 처리할 그래프 이름입니다. url은 원격 배포일 때 지정합니다. description은 언제 비동기로 위임해야 하는지 설명합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 deepagents에서 AsyncSubAgent, create_deep_agent를 준비합니다. 핵심 API는 create_deep_agent입니다. 노트북 기준 위치는 11_async_subagents.ipynb 노트북 기준입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

85. 슈퍼바이저에게는 비동기 작업을 관리하는 5가지 도구가 노출됩니다

슈퍼바이저에게는 비동기 작업을 관리하는 5가지 도구가 노출됩니다. 핵심은 시작 직후 결과를 기다리지 않고 사용자에게 제어를 돌려주는 것입니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 create_async_task는 배경 작업 시작입니다. 판단 기준은 긴 리서치·코딩 작업 위임입니다. check_async_task는 특정 작업 상태 확인입니다. 판단 기준은 사용자가 진행 상황을 물을 때입니다. list_async_tasks는 작업 목록 조회입니다. 판단 기준은 여러 배경 작업 관리입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

---

86. 비동기 위임의 좋은 대화는 시작·반환·조회·조정으로 나뉩니다

비동기 위임의 좋은 대화는 시작·반환·조회·조정으로 나뉩니다. 작업을 만들자마자 check하지 않는 것이 UX와 비용 모두에 중요합니다. 흐름을 왼쪽에서 오른쪽으로 읽으면 START 단계에서는 create task를 다룹니다. RETURN 단계에서는 control를 다룹니다. CHECK 단계에서는 status를 다룹니다. ADJUST 단계에서는 message를 다룹니다. MERGE 단계에서는 result를

다룹니다. 비동기 서브에이전트는 빨리 답하려는 기술이 아니라 긴 작업을 대화 밖에서 안전하게 굴리는 기술입니다. 에이전트 수업에서 중요한 것은 각 단계를 별개의 기능으로 보는 것이 아니라, 입력과 출력이 다음 단계로 어떻게 넘어가는지 보는 것입니다.

---

87. 개념 근거와 하네스 모델

이제 REFERENCE 개념 근거와 하네스 모델으로 넘어갑니다. Deep Agents 개념은 LangChain 공식 Deep Agents 문서와 LangGraph 공식 런타임 설명을 기준으로 보강합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 여기서는 개념 설명의 근거를 정리합니다. 실무에서 프레임워크를 설명할 때는 개인적인 비유보다 공식 문서와 논문에 기대는 편이 안전합니다.

---

88. Deep Agents는 planning, filesystem, subagents, memory를 묶은 agent harness입니다

공식 설명에서 Deep Agents는 LangGraph 위의 하네스입니다. 파일 시스템, 컨텍스트 관리, 서브에이전트, 장기 메모리를 묶고, todo planning은 필요할 때 미들웨어로 추가합니다. 이 슬라이드는 최신 기준의 용어 정리를 위한 참조입니다.

---

89. Backend routing은 파일 도구 표면과 실제 저장소를 분리합니다

Backend routing은 파일 도구 표면과 실제 저장소를 분리합니다. Deep Agents 공식 backends 문서는 StateBackend, StoreBackend, FilesystemBackend, routes 기반 구성을 설명합니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 File tools는 ls, read_file, write_file, edit_file 같은 표면은 같습니다. Routes는 경로 prefix로 저장소를 바꿉니다. Policy는 저장소 선택은 보안·수명·삭제 정책입니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

---

90. Deep Agents 개념 보강 출처는 공식 문서와 LangGraph 런타임 설명을 기준으로 삼습니다

Deep Agents 개념 보강 출처는 공식 문서와 LangGraph 런타임 설명을 기준으로 삼습니다. Deep Agents는 논문보다 공식 API·개념 문서가 더 직접적인 근거입니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 Agent harness는 Deep Agents official overview입니다. 판단 기준은 filesystem/subagents/memory와 선택 planning 설명입니다. Backends는 Deep Agents official backends docs입니다. 판단 기준은 StateBackend/StoreBackend/FilesystemBackend 경계입니다. Skills는 Deep Agents official skills docs입니다. 판단 기준은 progressive disclosure와 skill loading입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

---

91. 01_introduction.ipynb

이제 CODE 01_introduction.ipynb로 넘어갑니다. 01_introduction.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 04_deepagents/01_introduction.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

---

92. 01_introduction 코드 01

이제 01_introduction 코드 01 슬라이드입니다. 노트북 위치는 01_introduction.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 deepagents를 준비합니다. print 출력은 학습용 확인 지

점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

93. 01_introduction 코드 02

이제 01_introduction 코드 02 슬라이드입니다. 노트북 위치는 01_introduction.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 deepagents에서 create_deep_agent, SubAgent, CompiledSubAgent; deepagents에서 FilesystemMiddleware, MemoryMiddleware, SubAgentMiddleware를 준비합니다. 핵심 API는 create_deep_agent입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

94. 01_introduction 코드 03

이제 01_introduction 코드 03 슬라이드입니다. 노트북 위치는 01_introduction.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

95. 01_introduction 코드 04

이제 01_introduction 코드 04 슬라이드입니다. 노트북 위치는 01_introduction.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 inspect를 준비합니다. 핵심 API는 create_deep_agent입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

96. 02_quickstart.ipynb

이제 CODE 02_quickstart.ipynb로 넘어갑니다. 02_quickstart.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 04_deepagents/02_quickstart.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

---

97. 02_quickstart 코드 01

이제 02_quickstart 코드 01 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 3이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; os를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

98. 02_quickstart 코드 02

이제 02_quickstart 코드 02 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

99. 02_quickstart 코드 03

이제 02_quickstart 코드 03 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 6이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 deepagents에서 create_deep_agent; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 create_deep_agent, ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

100. 02_quickstart 코드 04

이제 02_quickstart 코드 04 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

101. 02_quickstart 코드 05 (1/2)

이제 02_quickstart 코드 05 (1/2) 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 typing에서 Literal; tavily에서 TavilyClient를 준비합니다. 함수 internet_search는 실제 작업 단위입니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

102. 02_quickstart 코드 05 (2/2)

이제 02_quickstart 코드 05 (2/2) 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

103. 02_quickstart 코드 06

이제 02_quickstart 코드 06 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 핵심 API는 create_deep_agent입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

104. 02_quickstart 코드 07

이제 02_quickstart 코드 07 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

105. 02_quickstart 코드 08 (1/2)

이제 02_quickstart 코드 08 (1/2) 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 21줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

106. 02_quickstart 코드 08 (2/2)

이제 02_quickstart 코드 08 (2/2) 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

107. 02_quickstart 코드 09

이제 02_quickstart 코드 09 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 12줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

108. 03_customization.ipynb

이제 CODE 03_customization.ipynb로 넘어갑니다. 03_customization.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 04_deepagents/03_customization.ipynb입니다. 슬라이

드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

---

109. 03_customization 코드 01

이제 03_customization 코드 01 슬라이드입니다. 노트북 위치는 03_customization.ipynb의 cell 2이고, 화면에는 실제 실행 코드 중 11줄이 보입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; deepagents에서 create_deep_agent를 준비합니다. 핵심 API는 create_deep_agent, ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

110. 03_customization 코드 02

이제 03_customization 코드 02 슬라이드입니다. 노트북 위치는 03_customization.ipynb의 cell 3이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

111. 03_customization 코드 03

이제 03_customization 코드 03 슬라이드입니다. 노트북 위치는 03_customization.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 핵심 API는 create_deep_agent입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

112. 03_customization 코드 04 (1/2)

이제 03_customization 코드 04 (1/2) 슬라이드입니다. 노트북 위치는 03_customization.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 핵심 API는 create_deep_agent, invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

113. 03_customization 코드 04 (2/2)

이제 03_customization 코드 04 (2/2) 슬라이드입니다. 노트북 위치는 03_customization.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘

기는지 봐야 합니다. 읽는 순서는 이렇습니다. 함수 get_user는 실제 작업 단위입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

114. 03_customization 코드 05 (1/3)

이제 03_customization 코드 05 (1/3) 슬라이드입니다. 노트북 위치는 03_customization.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 math를 준비합니다. 함수 calculate_compound_interest는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

115. 03_customization 코드 05 (2/3)

이제 03_customization 코드 05 (2/3) 슬라이드입니다. 노트북 위치는 03_customization.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 함수 convert_temperature는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

116. 03_customization 코드 05 (3/3)

이제 03_customization 코드 05 (3/3) 슬라이드입니다. 노트북 위치는 03_customization.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 핵심 API는 create_deep_agent입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

117. 03_customization 코드 06

이제 03_customization 코드 06 슬라이드입니다. 노트북 위치는 03_customization.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

118. 03_customization 코드 07 (1/2)

이제 03_customization 코드 07 (1/2) 슬라이드입니다. 노트북 위치는 03_customization.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 pydantic에서 BaseModel, Field를 준비합니다. 클래스 BookRecommendation, BookRecommendationList는 데이터 구조나 정책 단위를 정의합니다. 핵심 API는 create_deep_agent입니다. 설정값 response_format는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

119. 03_customization 코드 07 (2/2)

이제 03_customization 코드 07 (2/2) 슬라이드입니다. 노트북 위치는 03_customization.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 설정값 response_format는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

120. 03_customization 코드 08

이제 03_customization 코드 08 슬라이드입니다. 노트북 위치는 03_customization.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

121. 03_customization 코드 09

이제 03_customization 코드 09 슬라이드입니다. 노트북 위치는 03_customization.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 11줄이 보입니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

122. 04_backends.ipynb

이제 CODE 04_backends.ipynb로 넘어갑니다. 04_backends.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 04_deepagents/04_backends.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

123. 04_backends 코드 01

이제 04_backends 코드 01 슬라이드입니다. 노트북 위치는 04_backends.ipynb의 cell 2이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

124. 04_backends 코드 02

이제 04_backends 코드 02 슬라이드입니다. 노트북 위치는 04_backends.ipynb의 cell 3이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

125. 04_backends 코드 03

이제 04_backends 코드 03 슬라이드입니다. 노트북 위치는 04_backends.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 protocol에서 BackendProtocol를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

126. 04_backends 코드 04

이제 04_backends 코드 04 슬라이드입니다. 노트북 위치는 04_backends.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 12줄이 보입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 deepagents에서 create_deep_agent를 준비합니다. 핵심 API는 create_deep_agent, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

127. 04_backends 코드 05 (1/2)

이제 04_backends 코드 05 (1/2) 슬라이드입니다. 노트북 위치는 04_backends.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 pathlib에서 Path; tempfile를 준비합니다. 핵심 API는 create_deep_agent, invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

128. 04_backends 코드 05 (2/2)

이제 04_backends 코드 05 (2/2) 슬라이드입니다. 노트북 위치는 04_backends.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

129. 04_backends 코드 06

이제 04_backends 코드 06 슬라이드입니다. 노트북 위치는 04_backends.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 memory에서 InMemoryStore; memory에서 MemorySaver를 준비합니다. 핵심 API는 create_deep_agent, InMemoryStore입니다. 설정값 context는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

130. 04_backends 코드 07

이제 04_backends 코드 07 슬라이드입니다. 노트북 위치는 04_backends.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

131. 04_backends 코드 08

이제 04_backends 코드 08 슬라이드입니다. 노트북 위치는 04_backends.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

132. 04_backends 코드 09 (1/2)

이제 04_backends 코드 09 (1/2) 슬라이드입니다. 노트북 위치는 04_backends.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 핵심 API는 create_deep_agent, InMemoryStore입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

133. 04_backends 코드 09 (2/2)

이제 04_backends 코드 09 (2/2) 슬라이드입니다. 노트북 위치는 04_backends.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 2줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

134. 04_backends 코드 10

이제 04_backends 코드 10 슬라이드입니다. 노트북 위치는 04_backends.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 12줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 설정값 thread_id, configurable는

실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

135. 04_backends 코드 11 (1/4)

이 커스텀 백엔드 부록 코드는 현재 04_backends 노트북의 cell 19와 맞춘 버전입니다. ReadResult는 data 문자열이 아니라 file_data 딕셔너리와 줄 범위 메타데이터를 반환합니다. FileInfo는 딕셔너리처럼 path 키로 정렬하고, 예시 출력도 file_data content를 읽습니다.

---

136. 04_backends 코드 11 (2/4)

이 커스텀 백엔드 부록 코드는 현재 04_backends 노트북의 cell 19와 맞춘 버전입니다. ReadResult는 data 문자열이 아니라 file_data 딕셔너리와 줄 범위 메타데이터를 반환합니다. FileInfo는 딕셔너리처럼 path 키로 정렬하고, 예시 출력도 file_data content를 읽습니다.

---

137. 04_backends 코드 11 (3/4)

이 커스텀 백엔드 부록 코드는 현재 04_backends 노트북의 cell 19와 맞춘 버전입니다. ReadResult는 data 문자열이 아니라 file_data 딕셔너리와 줄 범위 메타데이터를 반환합니다. FileInfo는 딕셔너리처럼 path 키로 정렬하고, 예시 출력도 file_data content를 읽습니다.

---

138. 04_backends 코드 11 (4/4)

이 커스텀 백엔드 부록 코드는 현재 04_backends 노트북의 cell 19와 맞춘 버전입니다. ReadResult는 data 문자열이 아니라 file_data 딕셔너리와 줄 범위 메타데이터를 반환합니다. FileInfo는 딕셔너리처럼 path 키로 정렬하고, 예시 출력도 file_data content를 읽습니다.

---

139. 05_subagents.ipynb

이제 CODE 05_subagents.ipynb로 넘어갑니다. 05_subagents.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 04_deepagents/05_subagents.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

---

140. 05_subagents 코드 01

이제 05_subagents 코드 01 슬라이드입니다. 노트북 위치는 05_subagents.ipynb의 cell 2이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; os를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

141. 05_subagents 코드 02

이제 05_subagents 코드 02 슬라이드입니다. 노트북 위치는 05_subagents.ipynb의 cell 3이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

142. 05_subagents 코드 03

이제 05_subagents 코드 03 슬라이드입니다. 노트북 위치는 05_subagents.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

143. 05_subagents 코드 04 (1/2)

이제 05_subagents 코드 04 (1/2) 슬라이드입니다. 노트북 위치는 05_subagents.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 typing에서 Literal; tavily에서 TavilyClient를 준비합니다. 함수 internet_search는 실제 작업 단위입니다. 핵심 API는 create_deep_agent입니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

144. 05_subagents 코드 04 (2/2)

이제 05_subagents 코드 04 (2/2) 슬라이드입니다. 노트북 위치는 05_subagents.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

145. 05_subagents 코드 05

이제 05_subagents 코드 05 슬라이드입니다. 노트북 위치는 05_subagents.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 핵심 API는 create_deep_agent입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

146. 05_subagents 코드 06

이제 05_subagents 코드 06 슬라이드입니다. 노트북 위치는 05_subagents.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

147. 05_subagents 코드 07

이제 05_subagents 코드 07 슬라이드입니다. 노트북 위치는 05_subagents.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 deepagents에서 CompiledSubAgent를 준비합니다. 핵심 API는 create_deep_agent입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

148. 05_subagents 코드 08

이제 05_subagents 코드 08 슬라이드입니다. 노트북 위치는 05_subagents.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 핵심 API는 create_deep_agent입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

149. 05_subagents 코드 09 (1/2)

이제 05_subagents 코드 09 (1/2) 슬라이드입니다. 노트북 위치는 05_subagents.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 messages에서 HumanMessage; memory에서 MemorySaver를 준비합니다. 핵심 API는 create_deep_agent, invoke입니다. 설정값 thread_id, configurable, context는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

150. 05_subagents 코드 09 (2/2)

이제 05_subagents 코드 09 (2/2) 슬라이드입니다. 노트북 위치는 05_subagents.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 2줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

151. 05_subagents 코드 10 (1/2)

이제 05_subagents 코드 10 (1/2) 슬라이드입니다. 노트북 위치는 05_subagents.ipynb의 cell 19이고, 화면에는 실제 실행 코드 중 22줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 핵심 API는 create_deep_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

152. 05_subagents 코드 10 (2/2)

이제 05_subagents 코드 10 (2/2) 슬라이드입니다. 노트북 위치는 05_subagents.ipynb의 cell 19이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

153. 05_subagents 코드 11

이제 05_subagents 코드 11 슬라이드입니다. 노트북 위치는 05_subagents.ipynb의 cell 20이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

154. 06_memory_and_skills.ipynb

이제 CODE 06_memory_and_skills.ipynb로 넘어갑니다. 06_memory_and_skills.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 04_deepagents/06_memory_and_skills.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

155. 06_memory_and_skills 코드 01

이제 06_memory_and_skills 코드 01 슬라이드입니다. 노트북 위치는 06_memory_and_skills.ipynb의 cell 2이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; os를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

156. 06_memory_and_skills 코드 02

이제 06_memory_and_skills 코드 02 슬라이드입니다. 노트북 위치는 06_memory_and_skills.ipynb의 cell 3이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고

생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

157. 06_memory_and_skills 코드 03

이제 06_memory_and_skills 코드 03 슬라이드입니다. 노트북 위치는 06_memory_and_skills.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

158. 06_memory_and_skills 코드 04 (1/2)

이제 06_memory_and_skills 코드 04 (1/2) 슬라이드입니다. 노트북 위치는 06_memory_and_skills.ipynb의 cell 6이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 deepagents에서 create_deep_agent; backends에서 StateBackend, StoreBackend, CompositeBackend를 준비합니다. 핵심 API는 create_deep_agent, InMemoryStore입니다. 설정값 context는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

159. 06_memory_and_skills 코드 04 (2/2)

이제 06_memory_and_skills 코드 04 (2/2) 슬라이드입니다. 노트북 위치는 06_memory_and_skills.ipynb의 cell 6이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 핵심 API는 create_deep_agent입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

160. 06_memory_and_skills 코드 05

이제 06_memory_and_skills 코드 05 슬라이드입니다. 노트북 위치는 06_memory_and_skills.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

161. 06_memory_and_skills 코드 06

이제 06_memory_and_skills 코드 06 슬라이드입니다. 노트북 위치는 06_memory_and_skills.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 설정값

thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

162. 06_memory_and_skills 코드 07

이제 06_memory_and_skills 코드 07 슬라이드입니다. 노트북 위치는 06_memory_and_skills.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 tempfile를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

163. 06_memory_and_skills 코드 08

이제 06_memory_and_skills 코드 08 슬라이드입니다. 노트북 위치는 06_memory_and_skills.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 13줄이 보입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 핵심 API는 create_deep_agent, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

164. 06_memory_and_skills 코드 09

이제 06_memory_and_skills 코드 09 슬라이드입니다. 노트북 위치는 06_memory_and_skills.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 핵심 API는 create_deep_agent입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

165. 06_memory_and_skills 코드 10 (1/2)

이제 06_memory_and_skills 코드 10 (1/2) 슬라이드입니다. 노트북 위치는 06_memory_and_skills.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 함수 process_data는 실제 작업 단위입니다. 핵심 API는 invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

166. 06_memory_and_skills 코드 10 (2/2)

이제 06_memory_and_skills 코드 10 (2/2) 슬라이드입니다. 노트북 위치는 06_memory_and_skills.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 1줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

167. 07_advanced.ipynb

이제 CODE 07_advanced.ipynb로 넘어갑니다. 07_advanced.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 04_deepagents/07_advanced.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

168. 07_advanced 코드 01

이제 07_advanced 코드 01 슬라이드입니다. 노트북 위치는 07_advanced.ipynb의 cell 2이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; os를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

169. 07_advanced 코드 02

이제 07_advanced 코드 02 슬라이드입니다. 노트북 위치는 07_advanced.ipynb의 cell 3이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

170. 07_advanced 코드 03

이제 07_advanced 코드 03 슬라이드입니다. 노트북 위치는 07_advanced.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

171. 07_advanced 코드 04

이제 07_advanced 코드 04 슬라이드입니다. 노트북 위치는 07_advanced.ipynb의 cell 6이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 deepagents에서 create_deep_agent; memory에서 MemorySaver를 준비합니다. 핵심 API는 create_deep_agent입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

172. 07_advanced 코드 05 (1/2)

이제 07_advanced 코드 05 (1/2) 슬라이드입니다. 노트북 위치는 07_advanced.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

173. 07_advanced 코드 05 (2/2)

이제 07_advanced 코드 05 (2/2) 슬라이드입니다. 노트북 위치는 07_advanced.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 1줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

174. 07_advanced 코드 06 (1/2)

이제 07_advanced 코드 06 (1/2) 슬라이드입니다. 노트북 위치는 07_advanced.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 먼저 import는 types에서 Command를 준비합니다. 핵심 API는 invoke, Command입니다. 설정값 thread_id는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

175. 07_advanced 코드 06 (2/2)

이제 07_advanced 코드 06 (2/2) 슬라이드입니다. 노트북 위치는 07_advanced.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 2줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

176. 07_advanced 코드 07 (1/2)

이제 07_advanced 코드 07 (1/2) 슬라이드입니다. 노트북 위치는 07_advanced.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 typing에서 Literal; tavily에서 TavilyClient를 준비합니다. 함수 internet_search는 실제 작업 단위입니다. 핵심 API는 create_deep_agent입니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

177. 07_advanced 코드 07 (2/2)

이제 07_advanced 코드 07 (2/2) 슬라이드입니다. 노트북 위치는 07_advanced.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

178. 07_advanced 코드 08 (1/2)

이제 07_advanced 코드 08 (1/2) 슬라이드입니다. 노트북 위치는 07_advanced.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

179. 07_advanced 코드 08 (2/2)

이제 07_advanced 코드 08 (2/2) 슬라이드입니다. 노트북 위치는 07_advanced.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

180. 07_advanced 코드 09 (1/2)

이제 07_advanced 코드 09 (1/2) 슬라이드입니다. 노트북 위치는 07_advanced.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 21줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

181. 07_advanced 코드 09 (2/2)

이제 07_advanced 코드 09 (2/2) 슬라이드입니다. 노트북 위치는 07_advanced.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

182. 07_advanced 코드 10 (1/2)

이제 07_advanced 코드 10 (1/2) 슬라이드입니다. 노트북 위치는 07_advanced.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 tools에서 tool;

config에서 get_stream_writer를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 analyze_data는 실제 작업 단위입니다. 핵심 API는 stream입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

183. 07_advanced 코드 10 (2/2)

이제 07_advanced 코드 10 (2/2) 슬라이드입니다. 노트북 위치는 07_advanced.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

184. 07_advanced 코드 11 (1/2)

이제 07_advanced 코드 11 (1/2) 슬라이드입니다. 노트북 위치는 07_advanced.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 deepagents에서 create_deep_agent; langchain_anthropic에서 ChatAnthropic를 준비합니다. 핵심 API는 create_deep_agent, invoke입니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

185. 07_advanced 코드 11 (2/2)

이제 07_advanced 코드 11 (2/2) 슬라이드입니다. 노트북 위치는 07_advanced.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

186. 07_advanced 코드 12

이제 07_advanced 코드 12 슬라이드입니다. 노트북 위치는 07_advanced.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 deepagents에서 create_deep_agent; langchain_quickjs에서 CodeInterpreterMiddleware를 준비합니다. 핵심 API는 create_deep_agent입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

187. 07_advanced 코드 13 (1/2)

이제 07_advanced 코드 13 (1/2) 슬라이드입니다. 노트북 위치는 07_advanced.ipynb의 cell 20이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 acp에서 run_agent; deepagents에서 create_deep_agent를 준비합니다. 함수 main는 실제 작업 단위입니다. 핵심 API는 create_deep_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

188. 07_advanced 코드 13 (2/2)

이제 07_advanced 코드 13 (2/2) 슬라이드입니다. 노트북 위치는 07_advanced.ipynb의 cell 20이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

189. 07_advanced 코드 14

이제 07_advanced 코드 14 슬라이드입니다. 노트북 위치는 07_advanced.ipynb의 cell 22이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

190. 08_harness.ipynb

이제 CODE 08_harness.ipynb로 넘어갑니다. 08_harness.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 04_deepagents/08_harness.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

191. 08_harness 코드 01

이제 08_harness 코드 01 슬라이드입니다. 노트북 위치는 08_harness.ipynb의 cell 2이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; os를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

192. 08_harness 코드 02

이제 08_harness 코드 02 슬라이드입니다. 노트북 위치는 08_harness.ipynb의 cell 3이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

193. 08_harness 코드 03

이제 08_harness 코드 03 슬라이드입니다. 노트북 위치는 08_harness.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는

ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

194. 08_harness 코드 04

이제 08_harness 코드 04 슬라이드입니다. 노트북 위치는 08_harness.ipynb의 cell 6이고, 화면에는 실제 실행 코드 중 12줄이 보입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 설정값 configurable, context는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

195. 08_harness 코드 05

이제 08_harness 코드 05 슬라이드입니다. 노트북 위치는 08_harness.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 12줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

196. 08_harness 코드 06

이제 08_harness 코드 06 슬라이드입니다. 노트북 위치는 08_harness.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 12줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

197. 08_harness 코드 07

이제 08_harness 코드 07 슬라이드입니다. 노트북 위치는 08_harness.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

198. 08_harness 코드 08

이제 08_harness 코드 08 슬라이드입니다. 노트북 위치는 08_harness.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 설정값 configurable, context는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

199. 08_harness 코드 09 (1/2)

이제 08_harness 코드 09 (1/2) 슬라이드입니다. 노트북 위치는 08_harness.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 deepagents에서 create_deep_agent; langchain_quickjs에서 CodeInterpreterMiddleware를 준비합니다. 핵심 API

는 create_deep_agent입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

200. 08_harness 코드 09 (2/2)

이제 08_harness 코드 09 (2/2) 슬라이드입니다. 노트북 위치는 08_harness.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

201. 08_harness 코드 10

이제 08_harness 코드 10 슬라이드입니다. 노트북 위치는 08_harness.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

202. 08_harness 코드 11

이제 08_harness 코드 11 슬라이드입니다. 노트북 위치는 08_harness.ipynb의 cell 20이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

203. 09_comparison.ipynb

이제 CODE 09_comparison.ipynb로 넘어갑니다. 09_comparison.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 04_deepagents/09_comparison.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

---

204. 09_comparison 코드 01

이제 09_comparison 코드 01 슬라이드입니다. 노트북 위치는 09_comparison.ipynb의 cell 2이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; os를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

205. 09_comparison 코드 02

이제 09_comparison 코드 02 슬라이드입니다. 노트북 위치는 09_comparison.ipynb의 cell 3이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

206. 09_comparison 코드 03

이제 09_comparison 코드 03 슬라이드입니다. 노트북 위치는 09_comparison.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

207. 09_comparison 코드 04 (1/2)

이제 09_comparison 코드 04 (1/2) 슬라이드입니다. 노트북 위치는 09_comparison.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 22줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

208. 09_comparison 코드 04 (2/2)

이제 09_comparison 코드 04 (2/2) 슬라이드입니다. 노트북 위치는 09_comparison.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

209. 09_comparison 코드 05

이제 09_comparison 코드 05 슬라이드입니다. 노트북 위치는 09_comparison.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 21줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 함수 recommend_framework는 실제 작업 단위입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

210. 10_sandboxes_and_acp.ipynb

이제 CODE 10_sandboxes_and_acp.ipynb로 넘어갑니다. 10_sandboxes_and_acp.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은

04_deepagents/10_sandboxes_and_acp.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

---

211. 10_sandboxes_and_acp 코드 01

이제 10_sandboxes_and_acp 코드 01 슬라이드입니다. 노트북 위치는 10_sandboxes_and_acp.ipynb의 cell 2이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; os를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

212. 10_sandboxes_and_acp 코드 02

이제 10_sandboxes_and_acp 코드 02 슬라이드입니다. 노트북 위치는 10_sandboxes_and_acp.ipynb의 cell 3이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

213. 10_sandboxes_and_acp 코드 03

이제 10_sandboxes_and_acp 코드 03 슬라이드입니다. 노트북 위치는 10_sandboxes_and_acp.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

214. 10_sandboxes_and_acp 코드 04

이제 10_sandboxes_and_acp 코드 04 슬라이드입니다. 노트북 위치는 10_sandboxes_and_acp.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

215. 10_sandboxes_and_acp 코드 05 (1/2)

이제 10_sandboxes_and_acp 코드 05 (1/2) 슬라이드입니다. 노트북 위치는 10_sandboxes_and_acp.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습

니다. 먼저 import는 textwrap를 준비합니다. 핵심 API는 create_deep_agent, invoke입니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

216. 10_sandboxes_and_acp 코드 05 (2/2)

이제 10_sandboxes_and_acp 코드 05 (2/2) 슬라이드입니다. 노트북 위치는 10_sandboxes_and_acp.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

217. 10_sandboxes_and_acp 코드 06 (1/2)

이제 10_sandboxes_and_acp 코드 06 (1/2) 슬라이드입니다. 노트북 위치는 10_sandboxes_and_acp.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

218. 10_sandboxes_and_acp 코드 06 (2/2)

이제 10_sandboxes_and_acp 코드 06 (2/2) 슬라이드입니다. 노트북 위치는 10_sandboxes_and_acp.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

219. 10_sandboxes_and_acp 코드 07 (1/2)

이제 10_sandboxes_and_acp 코드 07 (1/2) 슬라이드입니다. 노트북 위치는 10_sandboxes_and_acp.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 acp에서 run_agent; deepagents에서 create_deep_agent를 준비합니다. 함수 main는 실제 작업 단위입니다. 핵심 API는 create_deep_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

220. 10_sandboxes_and_acp 코드 07 (2/2)

이제 10_sandboxes_and_acp 코드 07 (2/2) 슬라이드입니다. 노트북 위치는 10_sandboxes_and_acp.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

221. 10_sandboxes_and_acp 코드 08 (1/3)

이제 10_sandboxes_and_acp 코드 08 (1/3) 슬라이드입니다. 노트북 위치는 10_sandboxes_and_acp.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 acp에서 run_agent; deepagents에서 create_deep_agent를 준비합니다. 함수 main는 실제 작업 단위입니다. 핵심 API는 create_deep_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

222. 10_sandboxes_and_acp 코드 08 (2/3)

이제 10_sandboxes_and_acp 코드 08 (2/3) 슬라이드입니다. 노트북 위치는 10_sandboxes_and_acp.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

223. 10_sandboxes_and_acp 코드 08 (3/3)

이제 10_sandboxes_and_acp 코드 08 (3/3) 슬라이드입니다. 노트북 위치는 10_sandboxes_and_acp.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 2줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

224. 11_async_subagents.ipynb

이제 CODE 11_async_subagents.ipynb로 넘어갑니다. 11_async_subagents.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 04_deepagents/11_async_subagents.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

225. 11_async_subagents 코드 01

이제 11_async_subagents 코드 01 슬라이드입니다. 노트북 위치는 11_async_subagents.ipynb의 cell 2이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; os를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

226. 11_async_subagents 코드 02

이제 11_async_subagents 코드 02 슬라이드입니다. 노트북 위치는 11_async_subagents.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 deepagents에서 AsyncSubAgent, create_deep_agent를 준비합니다. 핵심 API는 create_deep_agent입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

227. 11_async_subagents 코드 03

이제 11_async_subagents 코드 03 슬라이드입니다. 노트북 위치는 11_async_subagents.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 이 코드는 Deep Agent 하네스 조립 방식을 다룹니다. 모델 호출 하나가 아니라 계획, 파일, 메모리, 서브에이전트가 함께 붙는 구조를 봅니다. 읽는 순서는 이렇습니다. 핵심 API는 create_deep_agent입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

228. 11_async_subagents 코드 04

이제 11_async_subagents 코드 04 슬라이드입니다. 노트북 위치는 11_async_subagents.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 pathlib에서 Path; json를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

229. 11_async_subagents 코드 05 (1/2)

이제 11_async_subagents 코드 05 (1/2) 슬라이드입니다. 노트북 위치는 11_async_subagents.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 21줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

230. 11_async_subagents 코드 05 (2/2)

이제 11_async_subagents 코드 05 (2/2) 슬라이드입니다. 노트북 위치는 11_async_subagents.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

231. Deep Agents 실전은 장기 작업을 제품 기능으로 만드는 단계입니다

Deep Agents 실전은 장기 작업을 제품 기능으로 만드는 단계입니다. 기초의 create_agent, 심화의 StateGraph 위에 파일·메모리·스킬·서브에이전트·샌드박스·ACP를 얹어 실전 하네스를 완성합니다. 이 트랙에서 남겨야 할 핵심은 계획, 파일시스템, 메모리, 서브에이전트, 스킬을 묶어 장기 작업 하네스를 설계하는 일입니다. 실습 코드를 모두 외울 필요는 없습니다. 대신 어떤 상황에

서 어떤 레이어를 선택하고, 상태와 도구와 권한을 어디에서 관리할지 말로 설명하는 수준까지 가야 합니다. 이 기준으로 다시 노트북을 열어 보면, 각 코드 셀이 단순 예제가 아니라 운영 가능한 에이전트를 구성하는 하나의 결정 지점으로 보입니다.